ASSIGNMENT ANSWERS

Introduction to Asymmetric Cryptography
Cryptography and Network Security

Q1. Basic principles of asymmetric (public-key) cryptography

Asymmetric cryptography is a method of securing information using a pair of mathematically related keys: a **public key** and a **private key**. The public key can be shared with everyone, while the private key must be kept secret by its owner. Data protected with one key can be verified or, in suitable schemes, reversed only with the matching key.

Roles of the keys

• **Public key:** Shared openly. It is used to encrypt data for the owner or to verify a digital signature made by the owner.
• **Private key:** Kept secret. It is used to decrypt data encrypted for its owner or to create a digital signature.

How it provides security

1. **Confidentiality:** Alice encrypts a message using Bob's public key. Only Bob's matching private key can decrypt it.
2. **Authentication:** Alice signs a message with her private key. Bob checks the signature using Alice's public key, helping confirm that it came from the holder of that private key.
3. **Non-repudiation:** A valid digital signature provides evidence that the signer's private key was used. This depends on secure key control, identity verification and sound procedures; it is not an absolute guarantee by itself.

```
Confidentiality process
ALICE (message) → encrypt with BOB'S PUBLIC KEY → encrypted message → BOB → decrypt with BOB'S PRIVATE KEY → original message
```

```
Signature process
ALICE → sign message using ALICE'S PRIVATE KEY → message + signature → BOB → verify using ALICE'S PUBLIC KEY → valid / invalid
```

Example: If Alice wants to send Bob a private document, she encrypts it with Bob's public key. If she also wants Bob to verify that she sent it, she can digitally sign it with her own private key.

Q2. Difference between symmetric and asymmetric cryptography

Basis	Symmetric cryptography	Asymmetric cryptography
Number of keys	One shared secret key is used for encryption and decryption.	A pair of keys is used: public key and private key.
Key distribution	The secret key must be shared securely before communication.	The public key can be distributed openly; the private key stays secret.
Speed	Generally very fast; suitable for large amounts of data.	Generally slower and more computationally expensive.
Key length	Usually shorter keys provide strong security (e.g. AES-128 or AES-256).	Usually much longer keys are needed (e.g. RSA-2048 or RSA-3072).
Scalability for n users	If every pair needs a separate key, n(n–1)/2 keys may be needed.	Each user can maintain one public/private key pair; public keys must be authenticated.
Typical applications	File and disk encryption, database encryption, and bulk data protection.	Key establishment, digital signatures, certificates, and identity authentication.

Note: Key-length numbers are not directly comparable between symmetric and asymmetric systems. The algorithms have different mathematical structures.

Q3. Main techniques and building blocks of public-key cryptography

1. Trapdoor one-way functions
A one-way function is easy to calculate in one direction but extremely difficult to reverse without special secret information. A trapdoor is the secret information that makes the reverse operation practical. RSA is based on the difficulty of factoring a large composite number, while its private key enables the corresponding decryption operation. These functions make public-key operations possible.

2. Diffie–Hellman key exchange
Diffie–Hellman allows two parties to establish a shared secret over a public channel. They agree on public parameters, choose private values, and exchange calculated public values. Each side combines the received value with its own private value to obtain the same shared secret. The secret can then be used to derive symmetric encryption keys. Basic Diffie–Hellman does not authenticate the parties by itself, so it must be combined with authentication to prevent a man-in-the-middle attack.

3. Digital signatures
The sender creates a signature using their private signing key, normally over a cryptographic hash of the message. The receiver uses the sender's public key to verify it. A valid signature supports message integrity, authentication of the signing key and non-repudiation evidence.

4. Public Key Infrastructure (PKI) and digital certificates
PKI is a system of policies, roles and technology for creating, distributing, validating and revoking public-key certificates. A digital certificate, commonly following the X.509 format, binds an identity to a public key and is digitally signed by a trusted Certificate Authority (CA). A receiver checks the CA signature, certificate validity period, identity and revocation status before trusting the key.

Contribution to secure communication: Trapdoor functions support secure key operations; Diffie–Hellman establishes shared secrets; signatures verify origin and integrity; and PKI helps users trust that a public key belongs to the claimed person or server.

Q4. Historical development of public-key cryptography

(a) Key distribution problem: Traditional symmetric cryptography requires both parties to possess the same secret key. If two people have never met, securely delivering that key is difficult. For a large network, managing a separate secret key for every pair also becomes complicated. This problem encouraged research into public-key methods.

(b) Diffie and Hellman, 1976: Whitfield Diffie and Martin Hellman published "New Directions in Cryptography." They introduced the concept of public-key cryptography and described a method for agreeing on a secret key over an insecure channel. Their work changed the direction of modern cryptography.

(c) RSA, 1977: Ron Rivest, Adi Shamir and Leonard Adleman developed the RSA public-key algorithm in 1977. RSA uses a public key for encryption or verification and a private key for decryption or signing. Its security is connected to the difficulty of factoring a large number into its prime factors.

(d) Earlier classified work at GCHQ: Researchers at the UK Government Communications Headquarters (GCHQ) independently developed related ideas before the public announcement. James Ellis developed the concept of non-secret encryption; Clifford Cocks devised an RSA-like system; and Malcolm Williamson developed a key-exchange method similar to Diffie–Hellman. This work remained classified for years and became publicly known later.

(e) Later schemes: Taher ElGamal introduced a public-key encryption/signature system in 1985, based on the discrete logarithm problem. Elliptic Curve Cryptography (ECC), developed during the 1980s, uses the mathematics of elliptic curves to provide strong security with comparatively small keys. ECC is widely used in modern key exchange and digital signatures.

Conclusion: Public-key cryptography solved important key-management problems and made secure communication, digital signatures and internet security practical.

Q5. Advantages and disadvantages of asymmetric cryptography

Advantages

• Solves much of the secret-key distribution problem because public keys can be shared openly.
• Supports digital signatures for integrity, authentication and non-repudiation evidence.
• Scales well in large networks because each user can maintain a key pair.
• Enables secure key establishment over an insecure network.
• PKI and certificates help bind public keys to identities.

Disadvantages

• Slower than symmetric encryption and unsuitable for directly encrypting very large files.
• Requires longer keys and more computational resources.
• Public keys must be authenticated; otherwise an attacker may substitute a false key.
• Private keys must be protected, backed up and managed carefully.
• Some algorithms may be threatened by sufficiently capable quantum computers.

Why hybrid cryptography is used in TLS

Real systems combine both approaches to get the benefits of each. During a TLS connection, the client and server authenticate the server using certificates and use an authenticated key-exchange method (commonly based on ephemeral Diffie–Hellman) to establish shared secret key material. They derive symmetric session keys and then use fast symmetric authenticated encryption, such as AES-GCM or ChaCha20-Poly1305, to protect the actual data.

Example: When a browser connects to an HTTPS website, public-key certificates and key exchange help authenticate the server and establish session keys. Symmetric encryption then efficiently protects pages, passwords and other data exchanged during that session.

Q6. RSA algorithm and key generation

RSA key generation steps

1. Choose two large, different prime numbers, **p** and **q**.
2. Calculate the modulus: **n = p × q**. The modulus is part of both keys.
3. Calculate Euler's totient: **ϕ(n) = (p − 1)(q − 1)**.
4. Choose a public exponent **e** such that 1 < e < ϕ(n) and gcd(e, ϕ(n)) = 1. A common choice is 65537.
5. Calculate the private exponent **d**, the modular inverse of e: **$d \times e \equiv 1 \pmod{\phi(n)}$**.
6. Public key is **(n, e)**; private key contains **(n, d)** and, in practice, the secret prime factors and other values for efficient operation.

Encryption

Represent the message as an integer m with 0 ≤ m < n. Compute the ciphertext: **$c = m^e \bmod n$**. The sender uses the receiver's public key.

Decryption

The receiver computes: **$m = c^d \bmod n$**, using the private key. In real applications, RSA encryption must use secure padding such as OAEP; raw textbook RSA must not be used to encrypt messages.

```
RSA flowchart
Choose primes p, q → Calculate n and ϕ(n) → Choose e → Find d → Publish (n,e), protect d →
Encode/pad message → Encrypt c = m^e mod n → Send c → Decrypt m = c^d mod n → Recover message
```

Why decryption works

The values e and d are chosen so that $ed = 1 + k\phi(n)$ for some integer k. For a message m relatively prime to n, Euler's theorem gives $m^{\phi(n)} \equiv 1 \pmod{n}$. Therefore, $c^d = (m^e)^d = m^{ed} = m^{1+k\phi(n)} \equiv m \pmod{n}$. RSA's correctness for all valid message representatives follows from the corresponding modular arithmetic modulo p and q (Chinese Remainder Theorem).

Small illustrative example (not secure): Let p=3 and q=11. Then n=33 and ϕ(n)=20. Choose e=3 and d=7 because 3×7=21 ≡ 1 (mod 20). For m=4, $c = 4^3 \bmod 33 = 31$; decrypting gives $31^{\blacksquare} \bmod 33 = 4$. Real RSA uses very large primes and padding.

Q7. RSA key sizes and security

Meaning of key size: RSA key size is the length of the modulus n = p×q, measured in bits. For example, a 2048-bit RSA key has a modulus represented using 2048 binary digits. A larger modulus generally makes factoring harder, but also increases computation and key size.

Why factoring matters: The public key reveals n. If an attacker can factor n into p and q, the attacker can calculate $\phi(n)$ and derive the private exponent d. Therefore, RSA security depends on choosing a sufficiently large modulus and keeping its prime factors secret.

Common sizes

- **1024-bit:** Considered too weak for new security-sensitive systems; should not be selected for modern deployments.
- **2048-bit:** Common minimum baseline for many current RSA applications.
- **3072-bit:** Provides a higher security margin and is commonly associated with about 128-bit classical security.
- **4096-bit:** Stronger against classical factoring than 2048-bit, but slower and not always necessary. It does not provide twice the security just because the bit length is doubled.

Approximate classical strength comparison

RSA-2048 ≈ 112-bit security ≈ ECC 224–255-bit range; RSA-3072 ≈ 128-bit security ≈ ECC-256; RSA-7680 ≈ 192-bit security ≈ ECC-384; RSA-15360 ≈ 256-bit security ≈ ECC-512. These are approximate equivalences, not exact conversions. ECC provides comparable classical security with much shorter keys.

Quantum-computer threat: Shor's algorithm, if run on a sufficiently large, fault-tolerant quantum computer, could factor RSA moduli efficiently and break RSA security. Such a computer is not currently available at the scale required to break properly sized RSA keys, but the future risk is important. Organisations should plan migration to standardised post-quantum cryptography for long-term protection.

Q8. RSA security considerations and attacks

(a) Factoring attacks: An attacker tries to factor n into p and q. If successful, the attacker can calculate the private key. Use a sufficiently large modulus generated from strong random prime numbers and follow current standards.

(b) Small public exponent and small-message attacks: A small exponent such as e=3 can be risky when raw, unpadded RSA is used and the message is small or repeated in a predictable way. Mathematical attacks may recover the message. Use standard padding such as OAEP and approved parameter choices; never use textbook RSA directly.

(c) Common modulus attack: If the same modulus n is improperly shared by different users with different public exponents, and an identical message is encrypted under certain conditions, an attacker may combine ciphertexts to recover the message. Each user should have independently generated RSA keys and modulus values.

(d) Timing and other side-channel attacks: These attacks use information leaked by implementation behaviour, such as execution time, power use, electromagnetic emissions or cache activity. Use constant-time implementations, secure cryptographic libraries, blinding for private-key operations, and hardware/software protections.

(e) Chosen-ciphertext attacks: An attacker submits selected ciphertexts and observes decryption-related responses. Poorly designed padding or error messages can leak information. RSA encryption should use OAEP and implementations should avoid distinguishable error responses and follow secure protocol designs.

Why OAEP padding is required

Raw RSA is deterministic: the same message encrypted with the same public key produces the same ciphertext. It is also vulnerable to mathematical and chosen-ciphertext attacks. Optimal Asymmetric Encryption Padding (OAEP) adds randomness and structured encoding before RSA encryption, helping provide semantic security when correctly implemented with an appropriate hash and mask-generation function. OAEP is specified for RSA encryption in PKCS #1.

RSA digital signatures

The signer calculates a cryptographic hash of the message and creates a signature using the private signing key with a standard signature encoding scheme, such as RSA-PSS. The receiver hashes the received message and verifies the signature using the signer's public key. If verification succeeds, it supports integrity and confirms that the signature corresponds to the private key. RSA-PSS is preferred for new RSA signature applications; encryption padding (OAEP) and signature padding (PSS) serve different purposes.

Good practices for secure RSA deployment

1. Use at least 2048-bit RSA for current general-purpose deployments; consider 3072-bit where a longer security horizon is required.
2. Generate keys using a cryptographically secure random number generator and a trusted library.
3. Never reuse one RSA modulus across users.
4. Use OAEP for RSA encryption and PSS for RSA signatures.
5. Protect private keys with access controls, secure storage and rotation procedures.
6. Use constant-time, maintained cryptographic implementations and avoid leaking detailed error messages.
7. Validate certificates and public-key identities; check expiry and revocation where applicable.
8. Keep software updated and plan for post-quantum migration.

References for further study

1. William Stallings, *Cryptography and Network Security: Principles and Practice*.
2. NIST Special Publication 800-57 Part 1, *Recommendation for Key Management*.
3. RFC 8017, *PKCS #1: RSA Cryptography Specifications Version 2.2*.
4. NIST FIPS 186-5, *Digital Signature Standard*.

Note: Answers are written in simple exam-oriented language. Key-size equivalences are approximate and security recommendations may change as standards evolve.